The Chain-of-Thought Journey

Five ways a transformer refused to run BFS, and what each one taught us

2026-07-03 → 07-05 · `src/cot.py`, `src/cot_tokens.py` · 2026-07-06

In one sentence. We wanted a shallow transformer to decide whether a graph is connected by *writing out* a breadth-first search, token by token — and getting there required five separate fixes, because five separate things silently sabotage a model that is supposed to learn an *algorithm* rather than a *statistic*.

1 The map

Every stage below produced a wrong result that *looked* like a different problem than it was. The journey, compressed:

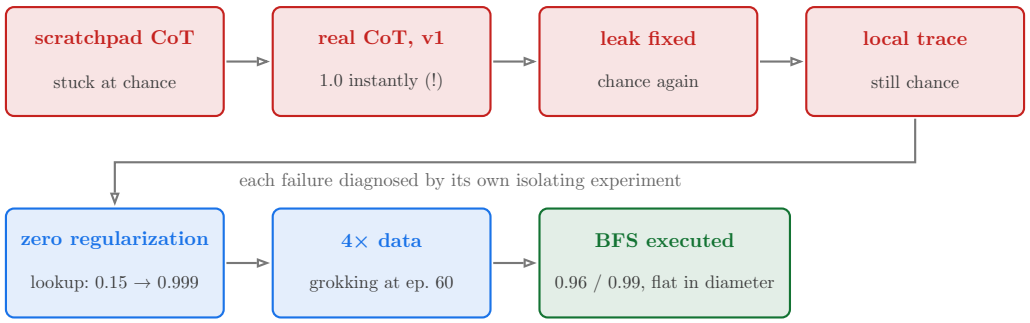

Figure 1: The route. Four configurations that failed for four different reasons, then two fixes that were about the *optimizer* and the *data*, not the architecture.

2 Background, from zero

2.1 The task, and why depth is the enemy

The task is binary **graph connectivity**: given all the edges, is there a path between every pair of nodes? The catch is *reachability distance*. Information in a transformer (or GNN) layer can only combine things that layer can see together: a message-passing layer moves information **one hop** along edges; even a global-attention layer needs on the order of $\log n$ layers to *compute* reachability (Sanford et al. 2024). So a model with a **fixed** number of layers has a fixed reasoning horizon — and a graph with a large diameter (longest shortest path) sits beyond it. Our datasets make this bite on purpose: caterpillar graphs with exact diameters 2–18, and an adversarial two-blob family where the connected and disconnected classes differ by **one single edge** with all simple statistics (degrees, edge counts) matched exactly.

2.2 What chain-of-thought is supposed to buy

A transformer that *generates text* is not depth-limited in the same way: after emitting a token, that token becomes **input** for the next step. Each generated token is one more sequential computation step — so a depth-2 model that emits d tokens has effectively bought d extra rounds (Merrill & Sabharwal 2024). The plan: make the model write out a breadth-first search —

```
prompt: N 0 1 2 .. E 0 3 1 4 ..  TRACE  completion: <the BFS> ANS YES EOS
```

— training it with **teacher forcing** and evaluating with **greedy decoding**.

Two words worth defining. *Teacher forcing*: during training the model always sees the **correct** trace so far and only predicts the next token — like doing a proof with the textbook open. *Greedy*

decoding: at test time it sees only its **own** previous tokens — the textbook is closed. A model can be excellent at the first and useless at the second; telling those apart is half of this document.

3 Act I — the scratchpad that was never chain-of-thought

What we built. Before true CoT, the repo's "CoT" was K *learnable scratchpad tokens* spliced into a single encoder pass, with an attention mask sketching "rounds": [vertices] + [edges] + $[c_1..c_K]$ + [task].

What we saw. Never better than the no-scratchpad baseline; both ≈ chance on the hard dataset (0.565 vs 0.63).

What was actually wrong. Four things, all structural — no amount of tuning could help:

1. **No supervision.** Only the task token gets a loss. Nothing ever tells c_k to hold "the nodes reachable in k hops"; that reading was a hope, not an objective.
2. **No content.** The c_i are the *same learned constants for every graph* — empty registers, not computed steps.
3. **Bypassable.** The task token attends to the graph directly, so the optimizer can — and measurably did — ignore the scratchpad entirely.
4. **Inert at depth 1.** Inside a single forward pass, c_2 can only see what c_1 *was*, not what c_1 *computed*, unless there is one layer per hop. The "sequential" chain silently requires the very depth it was meant to replace.

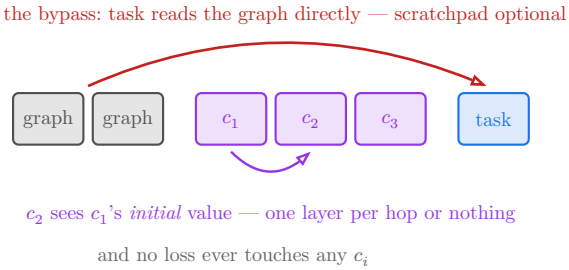

Figure 2: Why the scratchpad could not work: unsupervised, content-free, bypassable, and sequential only if depth pays for it anyway.

The fix. Replace it wholesale with a real autoregressive decoder (`cot_mode: autoregressive`): a supervised BFS trace as the target, tokens generated one at a time. **Why that works:** generation makes the steps *real* — each token is computed, written, and becomes input; supervision makes them *correct* — the loss grades every intermediate step, not just the verdict.

Lesson. CoT gains come from supervised intermediate steps. Unsupervised "thinking slots" in a single pass are just extra width wearing a costume.

4 Act II — the perfect score that was a leak

What we saw. The very first real CoT run: decoded accuracy **1.0 by epoch 5**. Champagne? No: trace exact-match was 0.000 and the out-of-distribution probe scored 0.37. The model answered perfectly *without ever producing a correct trace.*

What was actually wrong. In the caterpillar generator, a connected graph is one spanning tree ($n-1$ edges) and a disconnected one is two trees ($n-2$ edges). **Edge count was the label.** No GNN ever noticed — edge count isn't in their features. But a token model's prompt lists every edge, so the connected prompt is exactly **2 tokens longer**, and the learned **positional embedding** of the `TRACE` token differs between classes. The model read the answer off *where in the sequence it was standing.*

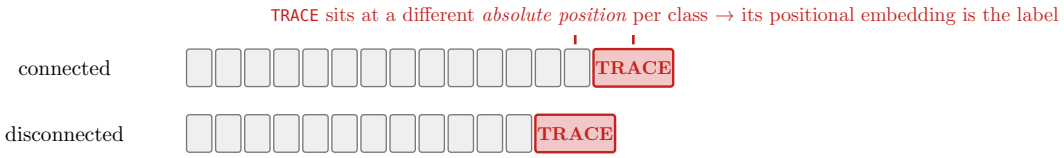

Figure 3: The sequence-length leak: n−1 vs n−2 edges means the prompt length differs by 2 tokens, and a learned position table can read that directly.

The fix. Pad every graph to $n - 1 + k$ edges, $k \sim U\{1..3\}$ drawn *independently of the label*, using chords that provably cannot change the graph's exact diameter (a leaf connected to a backbone neighbour of its own attachment; property-tested on 500 random caterpillars). **Why that works:** it removes the observable — after the fix, edge count (and hence sequence length) carries zero information about the class, so the only path to the answer runs through actual reachability.

Lesson. Every model family needs its own leak audit. Changing the representation (graph → token sequence) creates observables that never existed before — sequence length first among them. A perfect score that arrives too early is a symptom, not a success.

5 Act III — format learns, computation doesn't

What we saw. Leak fixed, chance again — but a *strange* chance. The loss froze at 1.87 (depths 1 and 2, full data, 20k steps). Teacher-forced answer accuracy was 1.0, decoded accuracy 0.5, trace exact-match 0. A per-position diagnostic (bucket every trace position by what it is, measure teacher-forced accuracy per bucket) localized it brutally: the model had learned *everything about how a trace looks* — separators 0.88, answer-reading 1.00, stopping 1.00 — and *nothing about what goes in it*: retrieving the neighbours of node 0 from the edge list sat at **5.6%**, with the whole correct prefix handed to it.

What was actually wrong. Two stacked problems. First, the known *next-token pitfall* (Bachmann & Nagarajan 2024): teacher forcing lets the model earn almost all its loss reduction from easy, statistical conditionals, and the one hard computation gets a weak gradient with no partial credit. Second, our compact trace made the hard part *maximally* hard: each BFS level was written as a **sorted set**, so its very first token is the **minimum of the whole frontier** — to be graded correct you must already have done the entire level's computation. All-or-nothing gradients on a global set operation.

The fix. A verbose trace format (`bfs_expand`) in which every next token is computable from *local* context:

compact (`bfs_levels`): `0 | 3 7 | 5 12 14 | ...`

first token of a level = **min of the whole frontier** — global computation, graded all-or-nothing

verbose (`bfs_expand`): `0 | EXP 0 3 7 | EXP 3 12 EXP 7 5 14 | ...`

parent after `EXP` = *copy* of previous level (induction); children = *lookup* keyed by the parent token right before them

Figure 4: The same BFS, two gradings. The verbose format costs 2× the tokens and buys a learnable curriculum: each next token is one small, locally-checkable step.

Why that works: gradient descent learns what the loss makes *locally* checkable. "Copy the previous level in order" is a textbook two-layer circuit (an induction head); "emit the neighbours of the token right before you" is one content lookup. The compact format demanded their composition *plus* a sort *plus* a set-minimum before the first token of a level could ever be right.

Lesson. A CoT trace is a curriculum, not a log. Design every next token to be computable from local context, or the gradient never finds the algorithm — it will learn the formatting and stop.

6 Act IV — the optimizer was the saboteur

What we saw. The verbose format *still* plateaued. Even pure copying sat at 0.119 after 20k steps at depth 2 — and copying is the easiest relational skill a transformer has. At this point the suspects were exotic (distractor tokens? embedding tying? attention numerics?). The scientific move was to shrink the task to one atom: a probe (`bfs_l1`) whose entire completion is *the sorted neighbours of node 0* — one lookup, nothing else — and then to turn knobs one at a time.

What was actually wrong. The first, cheapest knob was the answer. With the config's `weight_decay: 0.01` and `dropout: 0.1` the probe plateaued at trace-EM ≈ 0.15. Setting **both to zero** — no other change — took it to **0.95 by epoch 10** and 0.999 by 20:

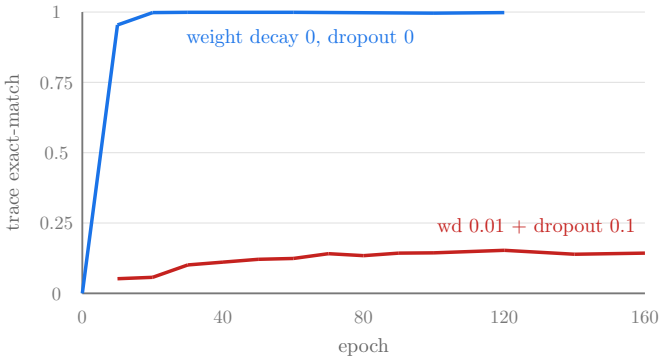

Figure 5: The atomic-lookup probe (`bfs_l1`, depth 2, same data, same everything). Zero regularization is the only difference between the two curves.

Why that happens. A retrieval circuit is a small set of *precise* weights: a key that must match a query exactly, an attention pattern that must be sharp. Weight decay (as Adam's L2 pull) shrinks all weights toward zero every step — for a big model a nuisance, for a 430k-parameter model with tied embeddings on a 42-token vocabulary it is a constant tax on exactly the weights the circuit needs, with no redundancy to pay it from. Dropout, meanwhile, injects noise into the very attention pattern that is trying to become sharp. Both settings are habits imported from *statistical* fitting, where they fight overfitting; here they fought *learning itself*. Notably this is the mirror image of the famous grokking result (Power et al. 2022), where weight decay **enables** delayed generalization — at our scale and task it **prevented** the circuit from ever forming.

Lesson. Regularizers tuned for statistical learning can be *the* blocker for algorithmic circuit formation at small scale. They are one `-o weight_decay=0 -o dropout=0` away from being ruled out — sweep them to zero before redesigning anything.

7 Act V — circuits versus memorization, an economics problem

What we saw. Zero regularization, full trace, depth 2, 8k graphs: the loss finally *collapsed* — and decoding still failed. The diagnostic showed a partial success with a clean edge: copying 0.989 and first-level lookup 0.995 (on unseen graphs!), but deeper levels — "neighbours of this parent *minus everything already visited*" — stuck at 0.6. Under exact-match, 0.6 per token compounds to 0 per trace.

The intuitive move, adding depth, backfired instructively: at depth 4 the training loss went to *literally 0.0000* while decoded trace-EM **declined**. The extra capacity was spent memorizing 6,400 training sequences, not building the missing circuit. (Its by-diameter accuracy even inverted — the model faithfully read its own derailed traces as "disconnected", scoring 0.25 exactly where connected graphs live.)

What was actually wrong — and the fix. Nothing was "wrong": it was a price comparison. A memorizing solution costs weights proportional to the training set; a circuit costs a fixed amount. Because node ids are freshly permuted in every graph, data is memorization-hostile: quadruple it and memorization's price quadruples while the circuit's stays flat. At **32k graphs, depth 2** the balance tipped, with a textbook grokking curve — flat for 55 epochs, then a phase transition:

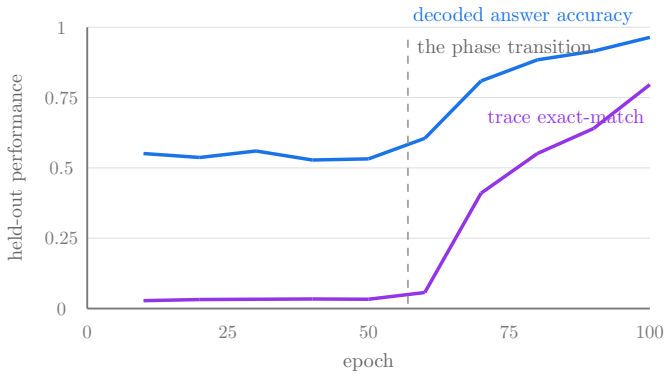

Figure 6: The grokking run: depth 2, 32k permuted graphs, zero regularization (`diameter_controlled`, run `20260705_005620`). Fifty flat epochs, then the circuit outcompetes memorization.

Lesson. Circuits vs memorization is an economics problem. Permuted-id data makes memorization expensive; capacity makes it cheap. When generalization stalls, add data before depth — extra layers may finance exactly the wrong solution.

8 The finish line

The same recipe then solved the *adversarial* dataset (`connectedness_hard_diam` — the one where the classes differ by a single edge and every single-pass architecture we ever tried sits at chance): decoded **0.9925**, trace exact-match **0.90**. The two summary pictures:

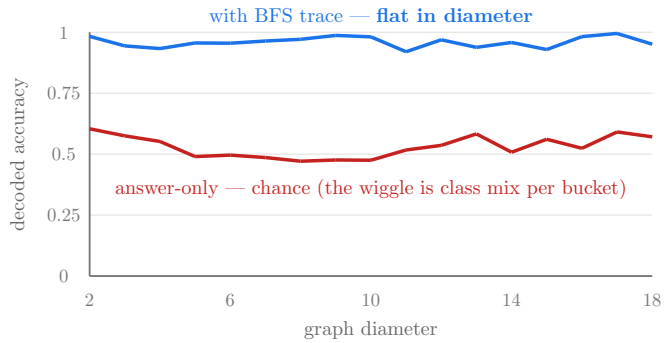

Figure 7: The thesis figure (`diameter_controlled`, 32k, depth 2): same model, same data, same optimizer — the trace is the only difference. Diameter, the quantity that defeats fixed depth, stops mattering when it is converted into tokens.

sub-circuit (teacher-forced, held-out)	**before**	**after**	**what it is**
parent after `EXP`	0.119	0.998	induction copy of the previous level
level-1 children	0.056	1.000	edge-list lookup
level-2+ children	~ 0.6	0.985–0.996	lookup minus the visited set
SEP / ANS / EOS	0.88–1.0	0.998–1.0	trace format & stopping

The mechanism receipts matter as much as the headline: the accuracy is carried by verified sub-circuits, not by an unexplained blob. And the honest caveat: all of this is **in-distribution** execution. The grokked caterpillar model fails on dense ER graphs (0.32); the blob-trained model does better (0.54) — a hint that *distribution diversity*, not the mechanism, is the remaining barrier. That is the open question this journey hands to the next one.

9 The checklist

For a fixed-depth transformer to execute a graph algorithm:

1. **Supervise the steps**, not just the answer — else: chance. (Act I)
2. **Audit the serialization for leaks**; permute everything permutable — else: a fake 1.0. (Act II)
3. **Make every next token locally computable** — else: format without content. (Act III)
4. **Zero regularization at small scale** — else: circuits never form. (Act IV)
5. **Enough distinct data that memorization costs more than the circuit** — else: perfect training loss, chance decoding. (Act V)
6. **Depth ≥ the per-step circuit** (two layers, for copy + lookup) — and no more than the data can discipline.

Every number is reproducible: configs `configs/cot_ar*.yaml`, run JSONs in `results/` (git provenance embedded), per-position diagnostics via `diag_cot_levels.py`, day-by-day narrative in `docs/CHANGELOG.md` (2026-07-03 → 07-05). Companion prose report: `reports/executing-graph-algorithms.md`.